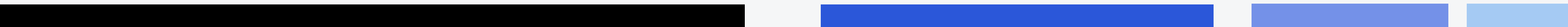


Lecture-1

Introduction



Instructions

- Please join **Google Classroom** to get updates.
- In case of any query, contact me on my email. (***ishabansatti@gmail.com***)
- Office: New Computer Lab (Mon to Thu: 10 AM to 1 PM)

Google Classroom Code



Marks Breakup

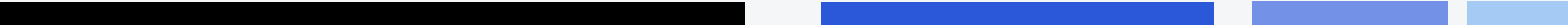
- Projects (30 = 3 X 10)
- Mid Exam (30)
- Final Exam (40)

Instructions

- You should have a GitHub account to upload your project codes.
- All projects will be followed by a relevant viva.
- Mid and Final Exams will be scenario based and may be open book subjected to approval.

Projects Details

Projects			
#	Project	Due	Marks
1	ETL + EDA Pipeline Build an ETL pipeline that extracts data from an API, transforms/validates it with pandas and Pydantic, loads it to a database, and conduct a structured exploratory data analysis on the result.	Week 7	10
2	scikit-learn ML Pipeline Build an end to end scikit-learn Pipeline combining preprocessing and a model, with cross-validated hyperparameter search, evaluated on a held-out test set.	Week 11	10
3	PyTorch + MLflow Train and tune a PyTorch model (using Optuna) and track all experiments, parameters, metrics, and the final model with MLflow.	Week 15	10
Total			30

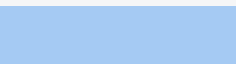


Tools Required for this course

- VS Code <https://code.visualstudio.com/> 
- Python <https://www.python.org/downloads/> 
- Docker <https://www.docker.com/> 
- GitHub Desktop <https://github.com/> 
- Google Colab <https://colab.research.google.com/> 

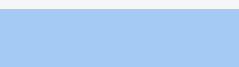
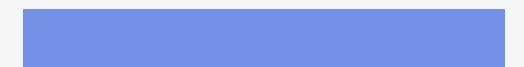
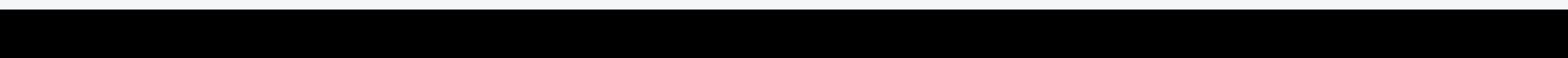
Agenda

01. Why Python?
02. Python's Typing Paradigm
03. Python Execution Engine and Interfaces
04. Variables
05. Input & Output
06. Type Conversion
07. Arithmetic Operations



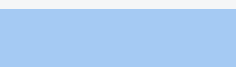
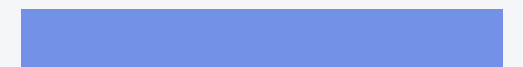
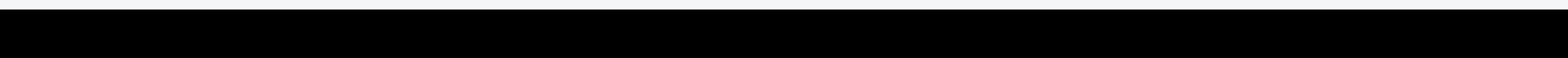
01.

Why Python?



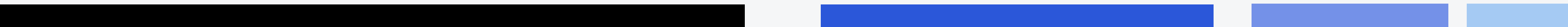
Python

- Default language for modern artificial intelligence and machine learning.
- **High-level**, and readable syntax.
- It acts as a thin, expressive wrapper around low-level performance engines written in C, and CUDA.
- Libraries such as PyTorch, NumPy, and Hugging Face Transformers expose Python APIs.

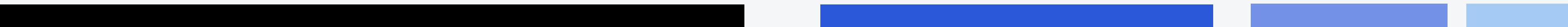


Python

- Enormous ecosystem: NumPy, pandas, PyTorch, TensorFlow, scikit-learn, Hugging Face.
- Rapid prototyping: dynamic typing and a **REPL**-friendly design.
- Community and education: the largest base of tutorials and courses.
- Interoperability: Python glues together C/C++ kernels, GPU drivers, distributed systems, and **web APIs** with minimal friction.

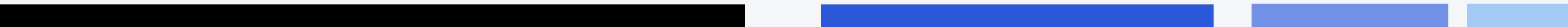


02. Typing Paradigms



Typing Paradigms

- Every programming language can be classified along two independent axes.
 - i. When types are checked: Static (at compile time) vs. Dynamic (at run time).
 - ii. How conversions between types are handled: Strong (no implicit coercion) vs. Weak (implicit coercion allowed).
- Python sits in a specific, important quadrant: **dynamically typed AND strongly typed.**



Static Typed

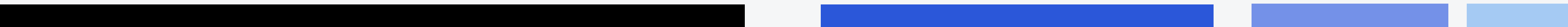
- Languages like C++, Java, and Rust require variable type to be declared and checked before the program runs.
- The compiler determines memory layout and allocations ***ahead of time (AOT)***, based on those declared types.
- **Benefit:** many type errors are caught early, before deployment, and the compiler can produce more optimized machine code.
- **Trade-off:** more verbose code, and less flexibility to rapidly reshape data structures during experimentation.
- **Example:** `int x = 42;` in C++ fixes x to the integer type for its entire lifetime.

Dynamic Typed

- In Python, a variable name is simply a label. It is a reference bound to an object living somewhere in memory.
- The type belongs to the OBJECT, not the variable name. A name can be rebound to point to any type of object at any time.
- Type checking happens dynamically, as **each instruction executes**.
- This enables extremely **fast prototyping**: you can reshape a variable's role without touching a type declaration.
- **The cost**: certain classes of bugs (e.g. calling a method that does not exist on the current object) are only caught when that exact line executes.

Weak Typed

- Weakly typed languages automatically convert (coerce) values between incompatible types when an operation is attempted.
- JavaScript example: `'5' + 5` silently evaluates to the string `'55'`.
- Convenient for quick scripting, but a frequent source of subtle bugs in large codebases.
- A hidden coercion can propagate incorrect values silently.

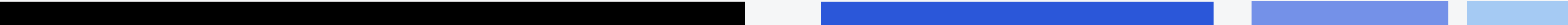


Strong Typed

- Python enforces type boundaries between fundamentally different types.
- Attempting an operation between incompatible types (e.g. `str + int`) raises an explicit `TypeError` at run time rather than silently coercing.
- This forces developers to be deliberate and explicitly convert types yourself, e.g. `str(98)` or `int("98")`.
- In AI code, this discipline prevents silent numerical corruption. **Imagine a batch size accidentally becoming a string and no error being raised.**

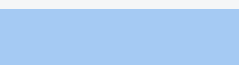
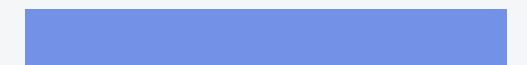
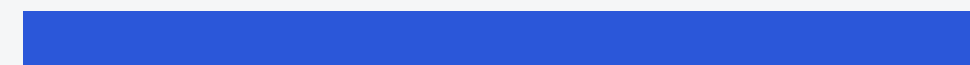
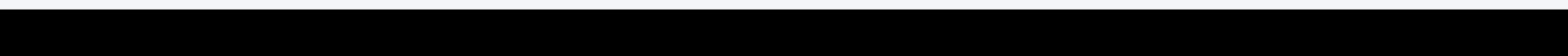
Typing Dimensions

Dimension	Dynamic (run-time check)	Static (compile-time check)
Strong (no implicit coercion)	Python, Ruby	Java, Rust, C#
Weak (implicit coercion)	JavaScript, PHP	C, C++



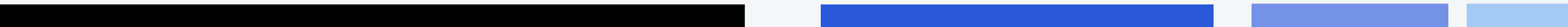
Question:

What happens at run time when you execute "Score: " + 98 in Python versus in JavaScript? Explain the underlying mechanism.



What happens at run time when you execute "Score: " + 98 in Python versus in JavaScript? Explain the underlying mechanism.

Python raises a `TypeError` because it is strongly typed and will not automatically coerce an integer to a string during addition. JavaScript is weakly typed and performs implicit coercion, yielding the string "Score: 98".

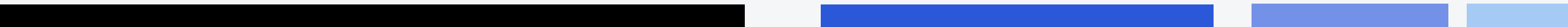


Question:

Which statement accurately describes Python's typing system?

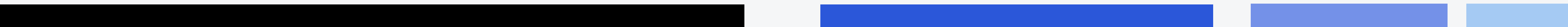
- ☐ **A** Statically and weakly typed
- ☒ **B** Dynamically and strongly typed
- ☐ **C** Dynamically and weakly typed
- ☐ **D** Statically and strongly typed

Answer: B *Python checks types at run time (dynamic) and refuses implicit coercion between incompatible types (strong).*



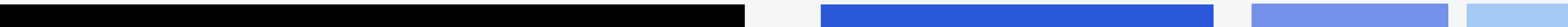
03.

Execution Engine and Interfaces



Execution Engine

- Every Python program needs an execution engine to run.
- CPython Interpreter: It reads source code, compiles it to bytecode, and executes that bytecode on a virtual machine.

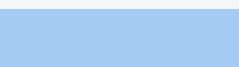
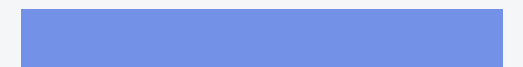
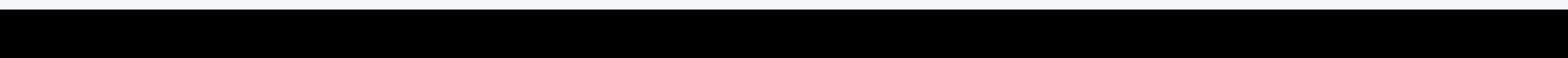


CPython Interpreter

1. **Parsing:** It reads your .py source file and checks for syntax errors, converting the text into an Abstract Syntax Tree (AST).
2. **Compiling to Bytecode:** The compiler component of CPython translates the AST into a low-level, platform-independent instruction set called **bytecode**.
3. **Caching (.pyc):** To speed up future runs, CPython writes this bytecode to disk inside a `__pycache__` directory as a .pyc file. If the source file hasn't changed on the next run, Python skips steps 1 and 2 entirely.
4. **Virtual Machine Execution:** The **CPython Virtual Machine (PVM)** loops through the bytecode instructions, evaluates them, and interacts with your system hardware to execute the program.

Interfaces

- REPL (Read-Eval-Print Loop): an interactive shell that executes one statement at a time and immediately shows results.
- Scripting: running a complete .py file from the command line, top to bottom, in one process.
- Notebook environments (IPython, Jupyter): cell-based, interactive execution popular for AI experimentation.



**Thank
You!**

